REST APIs

REST (Representational State Transfer) is **the architectural backbone of modern backend engineering**, enabling decoupled communication between clients (web, mobile, or AI agents) and server-side databases.

🧱 Core Constraints of REST Architecture

To be considered truly "RESTful", a backend API must adhere to six fundamental constraints originally defined by Dr. Roy Fielding:

Client-Server Separation: The frontend manages the user interface, while the backend handles data storage and business logic. This allows them to scale independently.

- **Client-Server Separation:** The frontend manages the user interface, while the backend handles data storage and business logic. This allows them to scale independently.

Statelessness: The server does not store any session context about the client. Every single incoming request must contain all the information necessary to understand and process it (e.g., authentication tokens).

- **Statelessness:** The server does not store any session context about the client. Every single incoming request must contain all the information necessary to understand and process it (e.g., authentication tokens).

Cacheability: Responses must explicitly define themselves as cacheable or non-cacheable to improve network efficiency and performance.

- **Cacheability:** Responses must explicitly define themselves as cacheable or non-cacheable to improve network efficiency and performance.

Uniform Interface: A standardized way of interacting with the server, which includes using logical resource URIs, standardized HTTP methods, and self-descriptive messages.

- **Uniform Interface:** A standardized way of interacting with the server, which includes using logical resource URIs, standardized HTTP methods, and self-descriptive messages.

Layered System: A client cannot tell whether it is connected directly to the end server or an intermediary, such as a load balancer or an **API Gateway**.

- **Layered System:** A client cannot tell whether it is connected directly to the end server or an intermediary, such as a load balancer or an **API Gateway**.

Code on Demand (Optional): The server can temporarily extend client functionality by transferring executable code (e.g., JavaScript scripts).

- **Code on Demand (Optional):** The server can temporarily extend client functionality by transferring executable code (e.g., JavaScript scripts).

🗺️ Resource Naming & URI Structure

In REST, everything is a resource. Backend developers design endpoints using clear, predictable naming patterns:

Action	HTTP Method	Endpoint (URI Path)	Description
Get All	`GET`	`/v1/users`	Retrieves a list of users
Get One	`GET`	`/v1/users/:id`	Retrieves a single user by their unique identifier
Create	`POST`	`/v1/users`	Creates a new user record
Replace	`PUT`	`/v1/users/:id`	Updates or completely replaces an entire user record
Modify	`PATCH`	`/v1/users/:id`	Applies a partial update to an existing user
Delete	`DELETE`	`/v1/users/:id`	Permanently removes a user

Key Rules for Endpoint Design:

Use Plural Nouns: Always target collections using nouns (`/products`, `/orders`) rather than verbs (`/getAllProducts`, `/createOrder`).

- **Use Plural Nouns:** Always target collections using nouns (`/products`, `/orders`) rather than verbs (`/getAllProducts`, `/createOrder`).

Apply Logical Nesting: Relate sub-resources hierarchically without diving too deep (e.g., `/v1/users/123/orders` to find orders belonging to a specific user).

- **Apply Logical Nesting:** Relate sub-resources hierarchically without diving too deep (e.g., `/v1/users/123/orders` to find orders belonging to a specific user).

API Versioning: Prefix routes with version tags (like `/v1/` or `/v2/`) to avoid breaking existing clients when introducing major database or payload alterations.

- **API Versioning:** Prefix routes with version tags (like `/v1/` or `/v2/`) to avoid breaking existing clients when introducing major database or payload alterations.

⚙️ How Data Travels through an API

Data moves between client and backend via four major plumbing mechanisms:

URL Path Parameters: Used strictly for identifying specific resources (e.g., the `123` in `/items/123`).

- **URL Path Parameters:** Used strictly for identifying specific resources (e.g., the `123` in `/items/123`).

Query Parameters: Key-value extensions following a `?` symbol, ideal for optional parameters like **filtering, sorting, and pagination** (e.g., `/products?category=shoes&page=2&limit=20`).

- **Query Parameters:** Key-value extensions following a `?` symbol, ideal for optional parameters like **filtering, sorting, and pagination** (e.g., `/products?category=shoes&page=2&limit=20`).

Request Body: The core payload container—typically formatted as **JSON**. Essential for `POST`, `PUT`, and `PATCH` requests. *Never pass sensitive data like passwords inside URL components; use the body instead.*

- **Request Body:** The core payload container—typically formatted as **JSON**. Essential for `POST`, `PUT`, and `PATCH` requests. *Never pass sensitive data like passwords inside URL components; use the body instead.*

HTTP Headers: Metadata strings detailing authentication credentials (e.g., `Authorization: Bearer <JWT>`) or payload formats (e.g., `Content-Type: application/json`).

- **HTTP Headers:** Metadata strings detailing authentication credentials (e.g., `Authorization: Bearer <JWT>`) or payload formats (e.g., `Content-Type: application/json`).

🚥 Standard HTTP Status Codes

Leveraging accurate status codes helps client applications elegantly handle successful operations and edge-case errors:

`200 OK`: The request succeeded, and data is returned.

- `200 OK`: The request succeeded, and data is returned.

`201 Created`: A new resource was successfully generated (common after a `POST`).

- `201 Created`: A new resource was successfully generated (common after a `POST`).

`204 No Content`: The request succeeded, but there is no payload to return (common after a `DELETE`).

- `204 No Content`: The request succeeded, but there is no payload to return (common after a `DELETE`).

`400 Bad Request`: The server cannot process the request due to client validation errors.

- `400 Bad Request`: The server cannot process the request due to client validation errors.

`401 Unauthorized`: Authentication is required or has failed.

- `401 Unauthorized`: Authentication is required or has failed.

`403 Forbidden`: The client is authenticated but does not have permission to access the resource.

- `403 Forbidden`: The client is authenticated but does not have permission to access the resource.

`404 Not Found`: The requested resource does not exist on the server.

- `404 Not Found`: The requested resource does not exist on the server.

`500 Internal Server Error`: The backend encountered an unhandled code crash or database failure.

- `500 Internal Server Error`: The backend encountered an unhandled code crash or database failure.

🛡️ Production Best Practices

When building enterprise-ready APIs, keep these production pillars in mind:

Idempotency: Ensure that calling an endpoint multiple times yields the exact same server state. `GET`, `PUT`, and `DELETE` should inherently be idempotent, whereas successive `POST` calls will repeatedly create duplicate resources.

- **Idempotency:** Ensure that calling an endpoint multiple times yields the exact same server state. `GET`, `PUT`, and `DELETE` should inherently be idempotent, whereas successive `POST` calls will repeatedly create duplicate resources.

Input Validation: Use tools like Pydantic (Python/FastAPI) or Zod (TypeScript) to sanitize and validate request payloads before they touch database schemas.

- **Input Validation:** Use tools like Pydantic (Python/FastAPI) or Zod (TypeScript) to sanitize and validate request payloads before they touch database schemas.

Rate Limiting: Protect backend computational resources from DDoS attacks or script abuse by throttling requests per IP address or user account.

- **Rate Limiting:** Protect backend computational resources from DDoS attacks or script abuse by throttling requests per IP address or user account.

Documentation: Keep documentation live and synchronized using the OpenAPI Specification (OAS) or automated tools like Swagger UI.

- **Documentation:** Keep documentation live and synchronized using the OpenAPI Specification (OAS) or automated tools like Swagger UI.